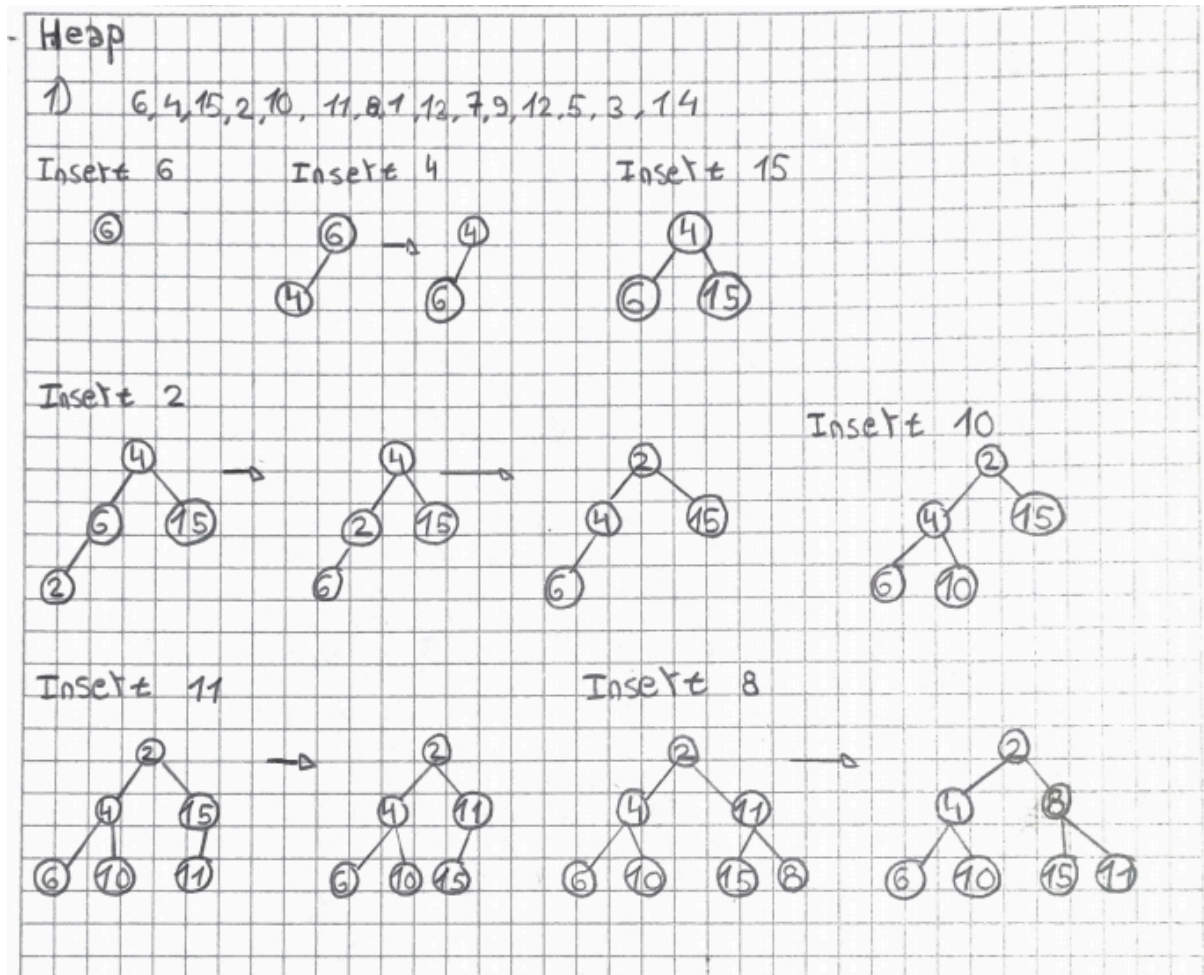


Ejercitación teórica de Heap

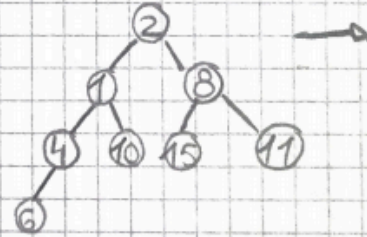
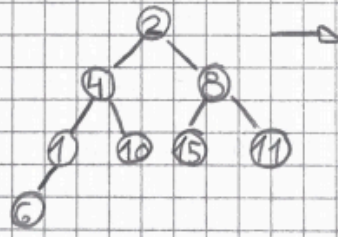
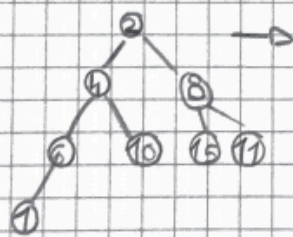
Lista de ejercitación

- ☒ 1
- ☒ 2
- ☒ 3
- ☒ 4
- ☒ 5
- ☒ 6
- ☒ 7
- ☒ 8
- ☒ 9
- ☒ 10
- ☒ 11
- ☐ 12
- ☐ 13
- ☐ 14
- ☐ 15
- ☐ 16
- ☐ 17
- ☐ 18
- ☐ 19
- ☐ 20
- ☐ 21
- ☐ 22

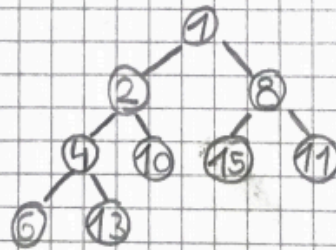
Ejercicio 1



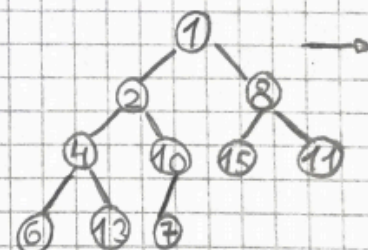
Insert 1



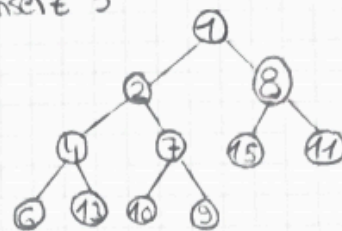
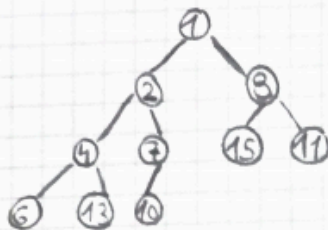
Insert 13



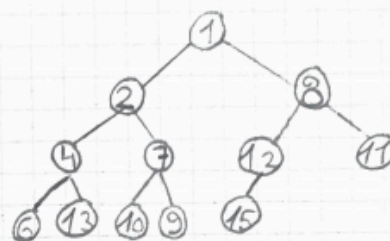
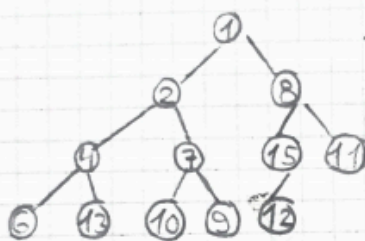
Insert 7



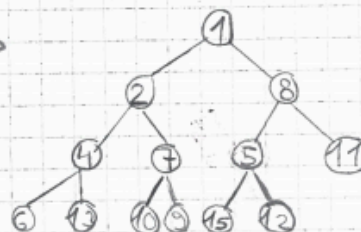
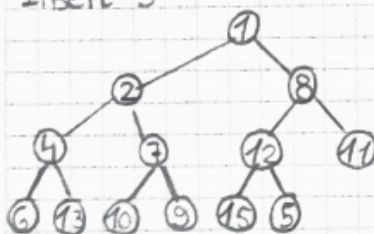
Insert 9

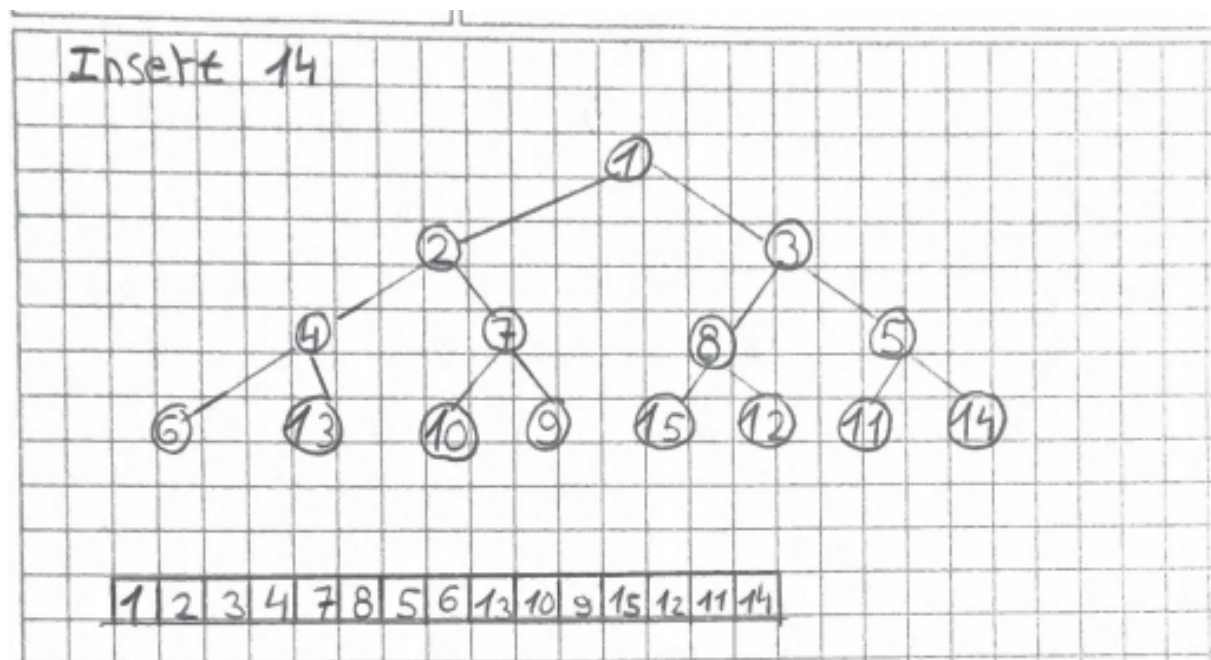
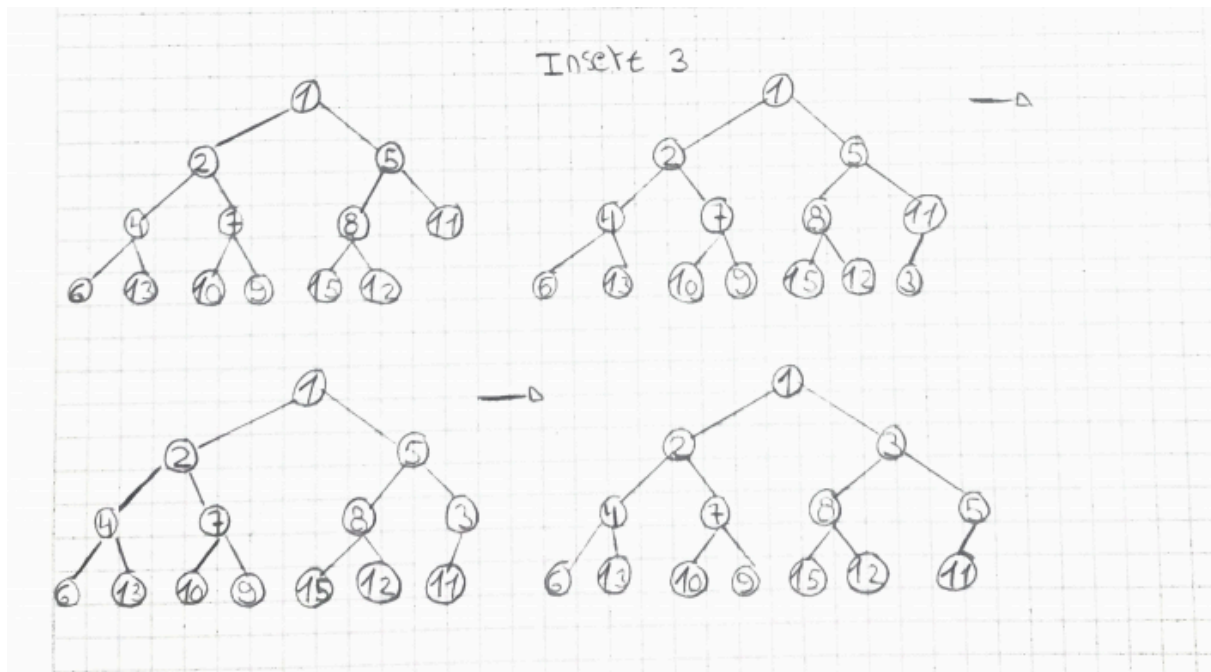


Insert 12



Insert 5





Ejercicio 2

a) ¿Cuántos elementos hay, al menos, en una heap de altura h ?

Acá nos viene bárbaro todo lo que dedujiste de árboles en la práctica anterior. Recordá que una heap es un árbol binario **completo**.

Para que alcance la altura h con la mínima cantidad de nodos, todos los niveles anteriores (del 0 al $h - 1$) tienen que estar completamente llenos, y el último nivel (h) tiene que tener solo 1 nodo.

- La suma de todos los niveles llenos desde 0 hasta $h - 1$ da siempre por fórmula: $2^h - 1$.
- A eso le sumamos el nodo solitario del nivel h : $(2^h - 1) + 1 = 2^h$.
- **Respuesta:** En una heap de altura h , hay al menos 2^h elementos.
(Si querés probarlo: en altura 3, el mínimo es $2^3 = 8$ nodos. ¡Exactamente lo que habías dibujado mal al principio y luego corregimos!)

b) ¿Dónde se encuentra ubicado el elemento mínimo en una max-heap?

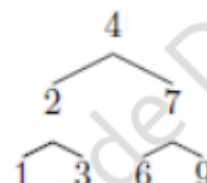
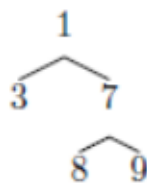
El elemento mínimo en una max-heap se encuentra ubicado en una de las hojas.

c) ¿El siguiente arreglo es una max-heap : [23, 17, 14, 6, 13, 10, 1, 5, 7, 12] ?

No, el arreglo no es una max-heap. Al acomodar el árbol de izquierda a derecha por niveles, todos los padres son mayores a sus respectivos hijos excepto el 7 que es mayor a su padre 6.

Ejercicio 3

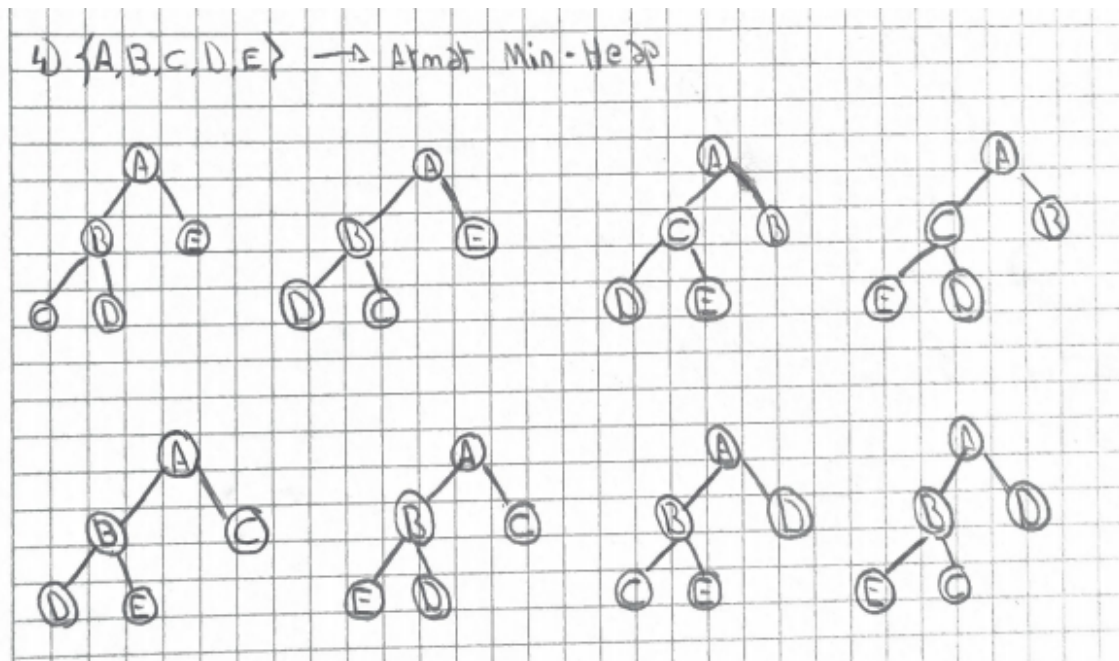
Dados los siguientes árboles, indique si representan una heap. Justifique su respuesta.



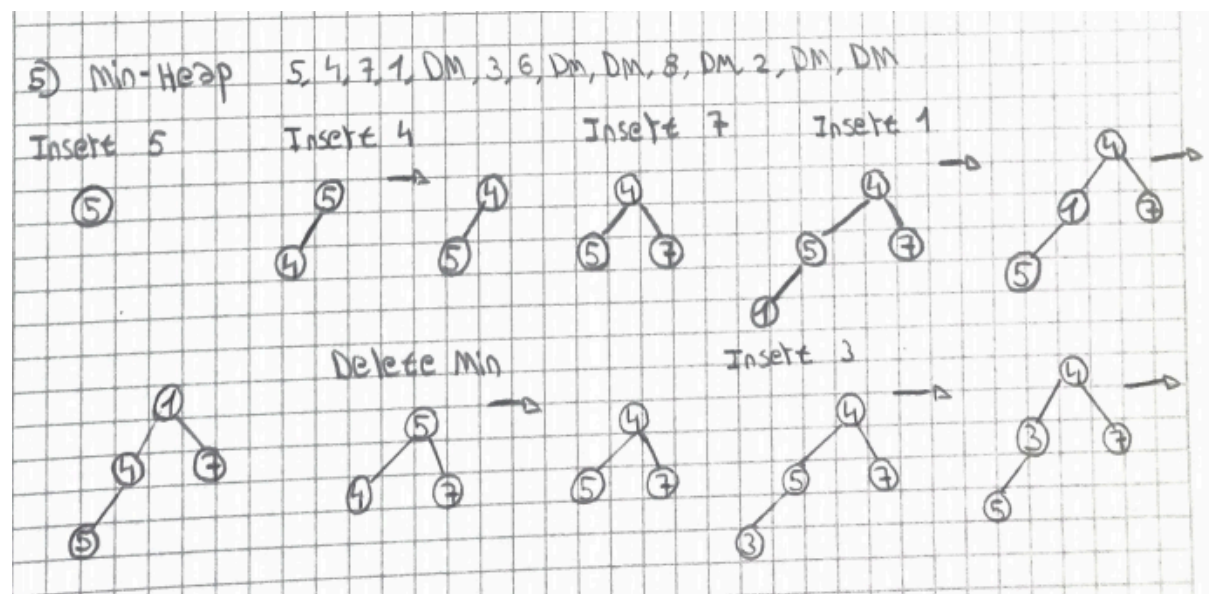
El primero no es una heap porque no es un árbol binario completo.

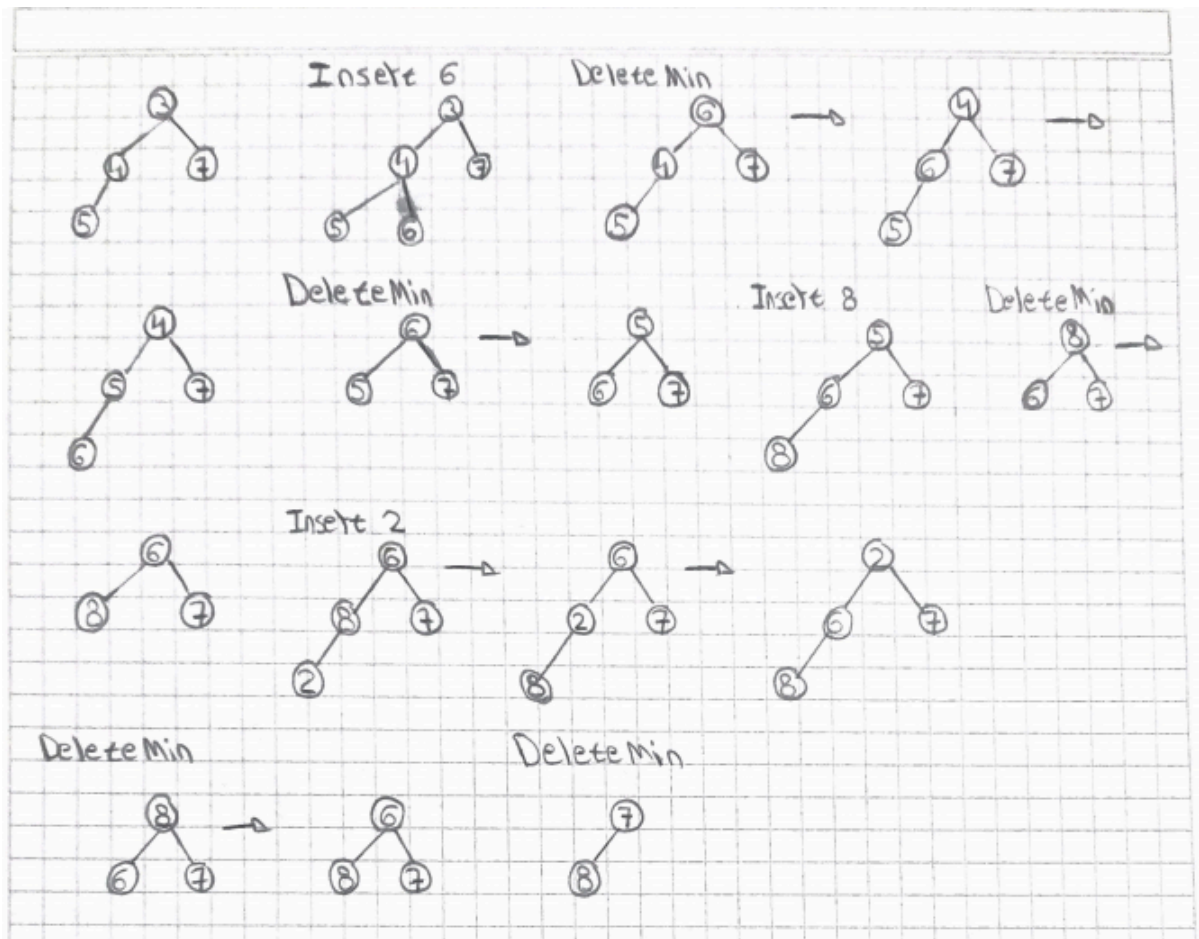
El segundo no es una heap porque no supera la prueba de orden. No es una max-heap porque no todos los padres son mayores a sus respectivos hijos, como tampoco es una min-heap porque no todos los padres son menores a sus respectivos hijos.

Ejercicio 4



Ejercicio 5

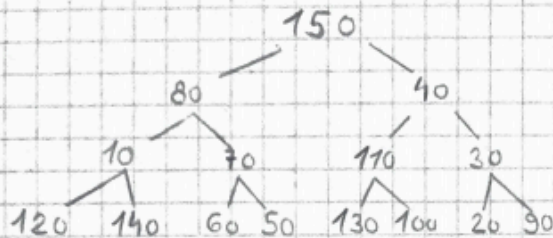




Ejercicio 6

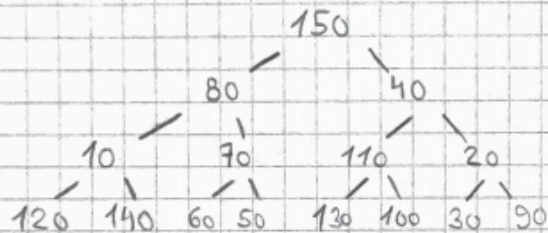
Q $\{150, 80, 40, 10, 70, 110, 30, 120, 140, 60, 50, 130, 100, 20, 90\}$

MinHeap

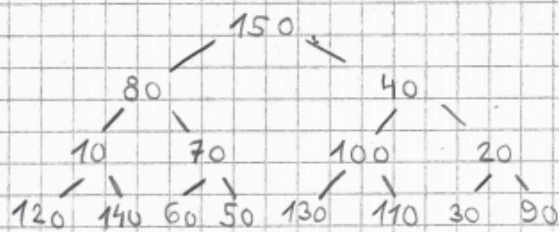


$i = \text{tamaño}(V) / 2 = 15 / 2 = 7 \rightarrow V[i] = V[7] = 30$

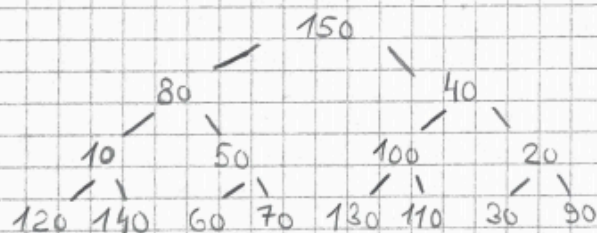
Nodo 7



Nodo 6

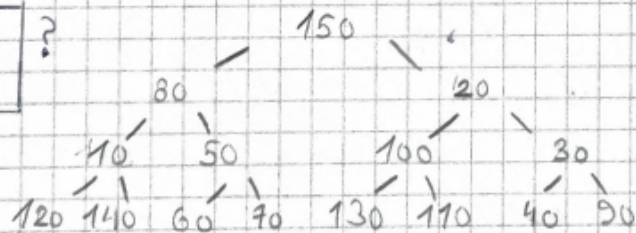


Nodo 5

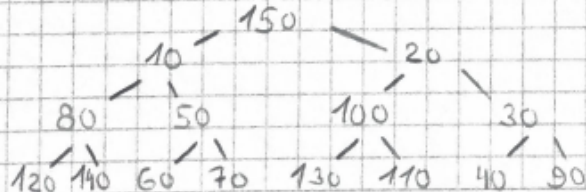


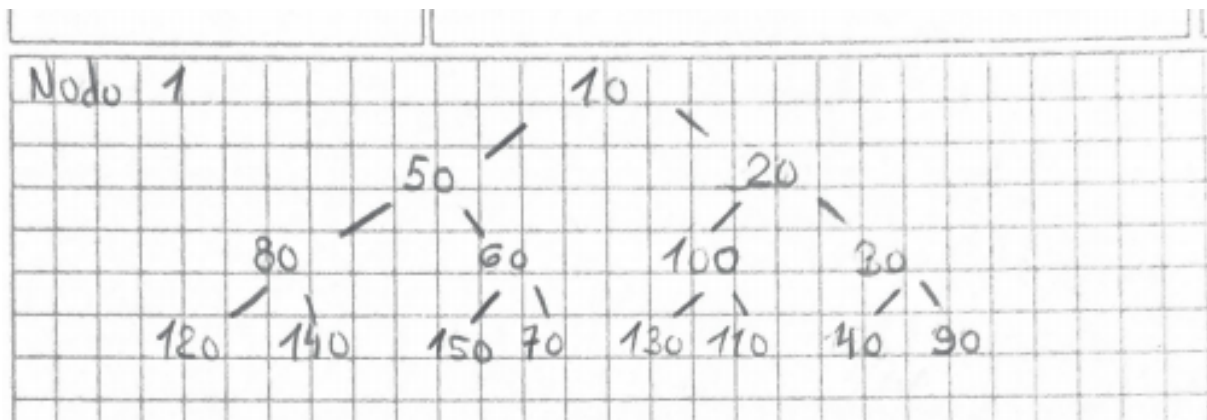
Nodo 4 ✓

Nodo 3



Nodo 2





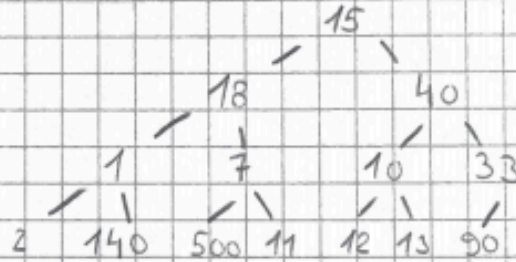
Ejercicio 7

7)

$V = \{15, 18, 40, 1, 7, 10, 33, 2, 140, 500, 11, 12, 13, 90\}$

$i = \text{tamaño}(V)/2 = 14/2 = 7$

MinHeap



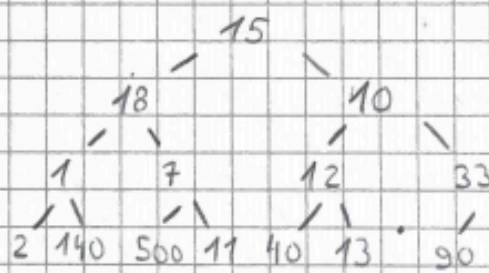
Nodo 7 ✓

Nodo 6 ✓

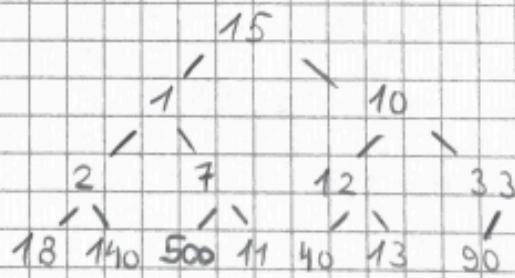
Nodo 5 ✓

Nodo 4 ✓

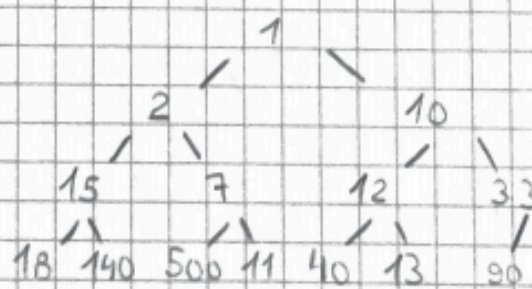
Nodo 3



Nodo 2

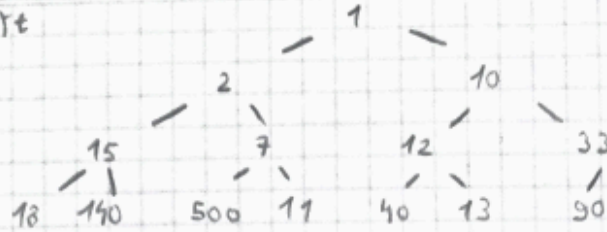


Nodo 1

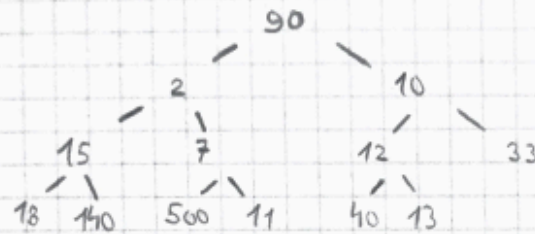


$\{1, 2, 10, 15, 7, 12, 33, 18, 140, 500, 11, 40, 13, 90\}$

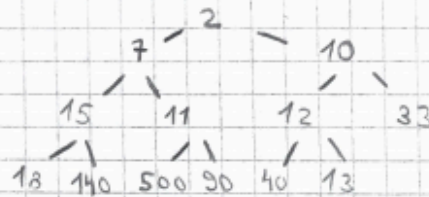
Heap Sort



Reemplazo



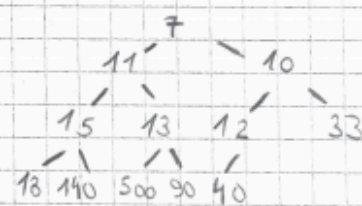
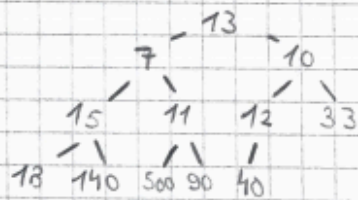
Filtro



{2, 7, 10, 15, 11, 12, 33, 18, 140, 500, 90, 40, 13 | 1}

Reemplazo

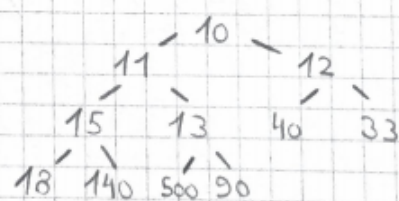
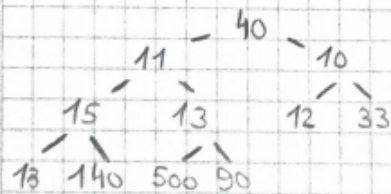
Filtro



{7, 11, 10, 15, 13, 12, 33, 18, 140, 500, 90, 40 | 2, 1}

Reemplazo

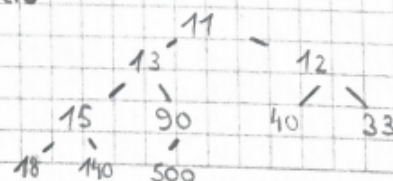
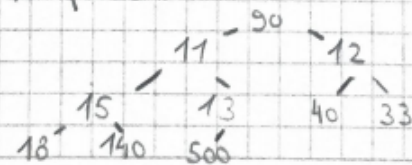
Filtro



{10, 11, 12, 15, 13, 40, 33, 18, 140, 500, 90 | 7, 2, 1}

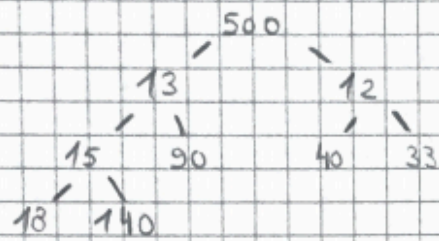
Reemplazo

Filtro

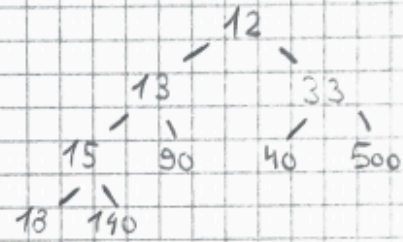


{11, 13, 12, 15, 90, 40, 33, 18, 140, 500 | 10, 7, 2, 1}

Reemplazo

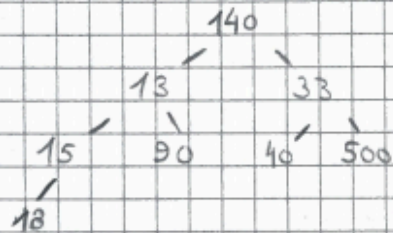


Filtro

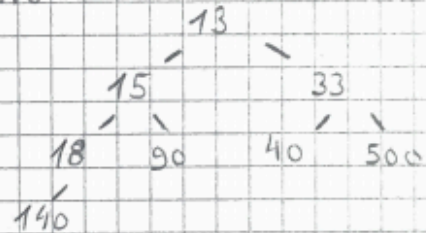


{12, 13, 33, 15, 90, 40, 500, 18, 140 | 11, 10, 7, 2, 1}

Reemplazo

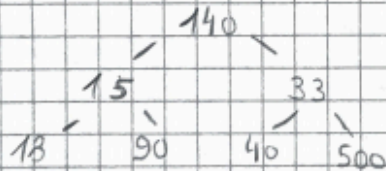


Filtro

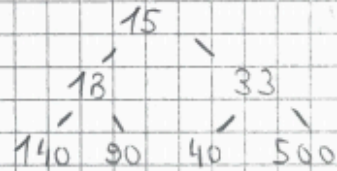


{13, 15, 33, 18, 90, 40, 500, 140 | 12, 11, 10, 7, 2, 1}

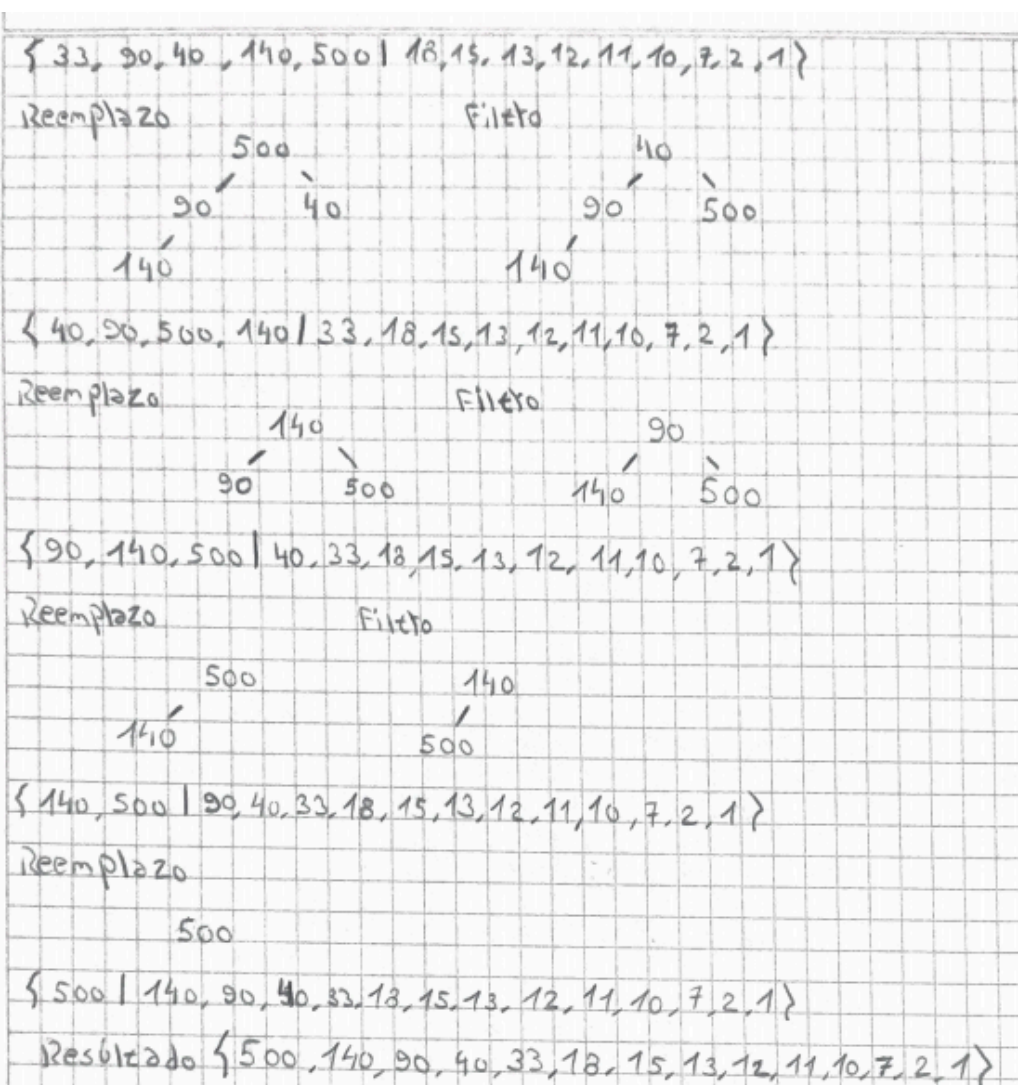
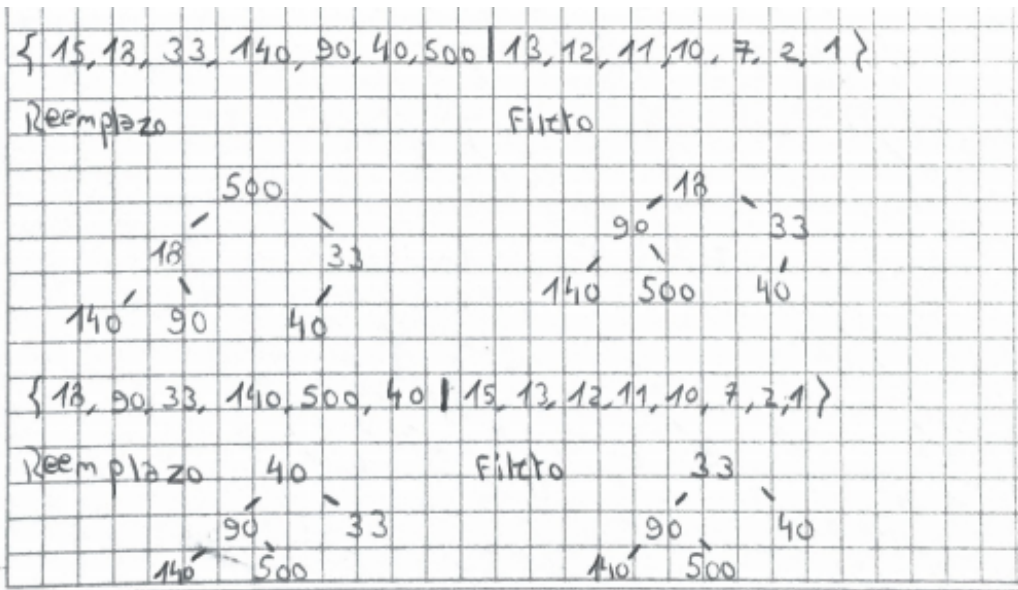
Reemplazo



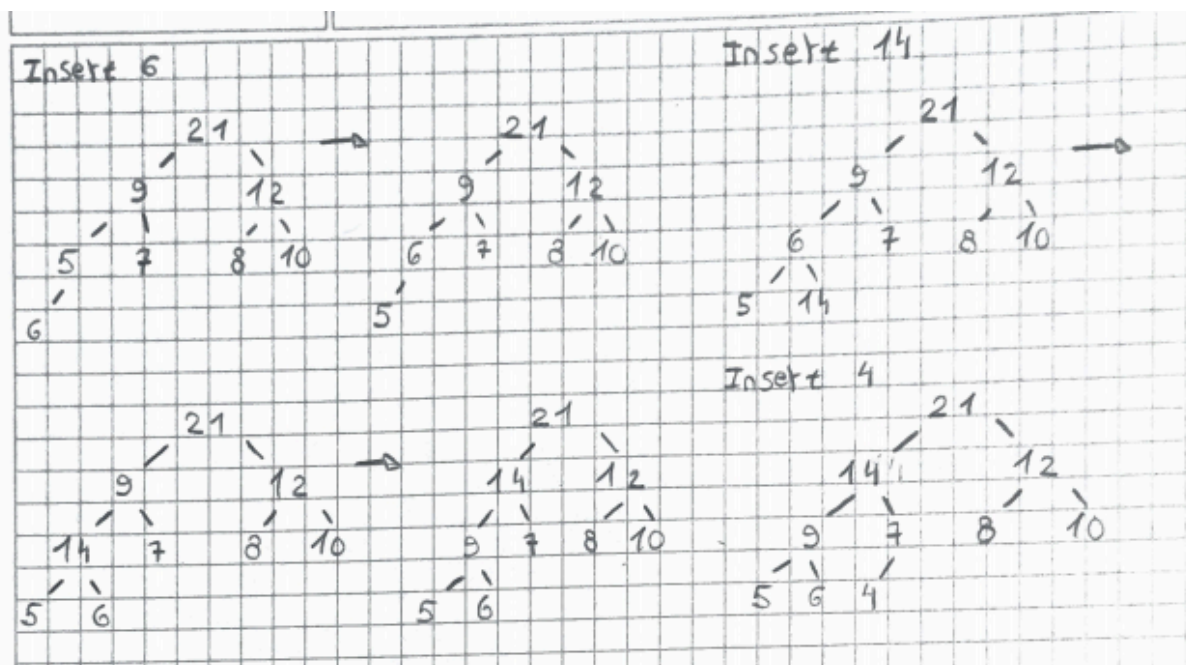
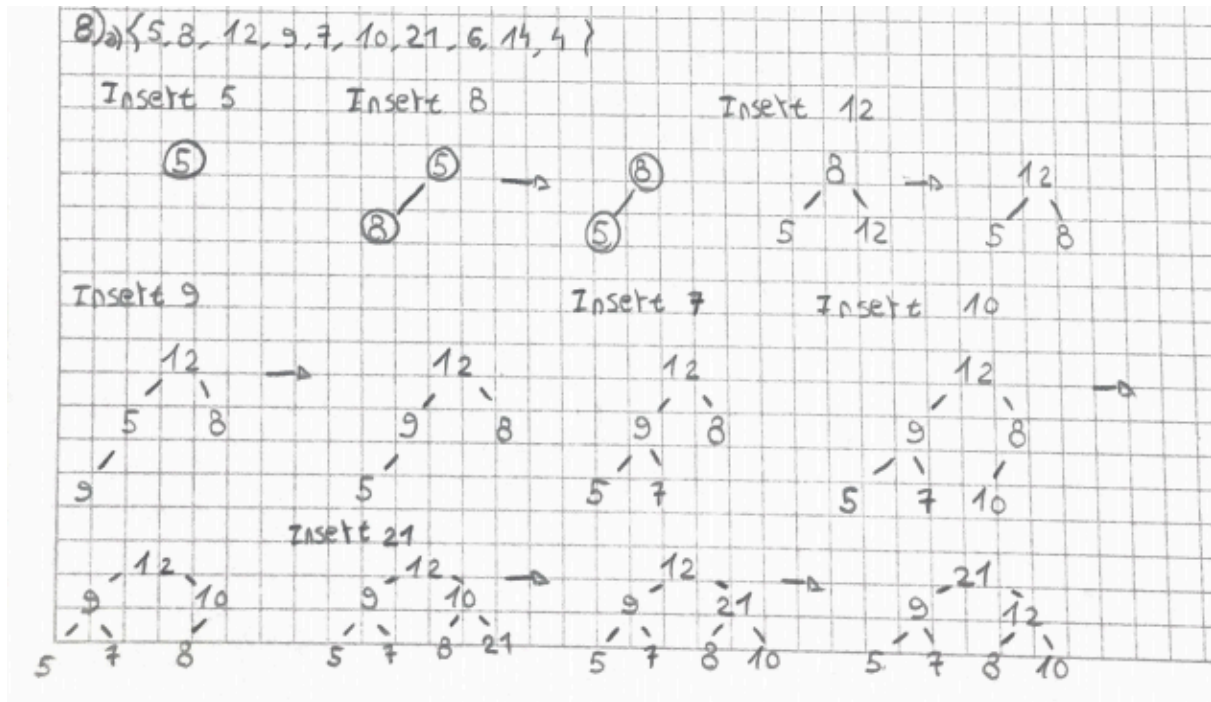
Filtro

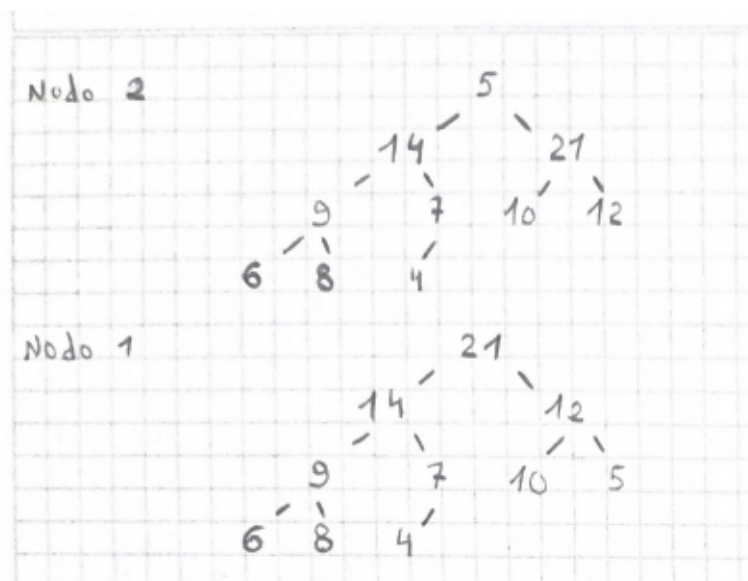
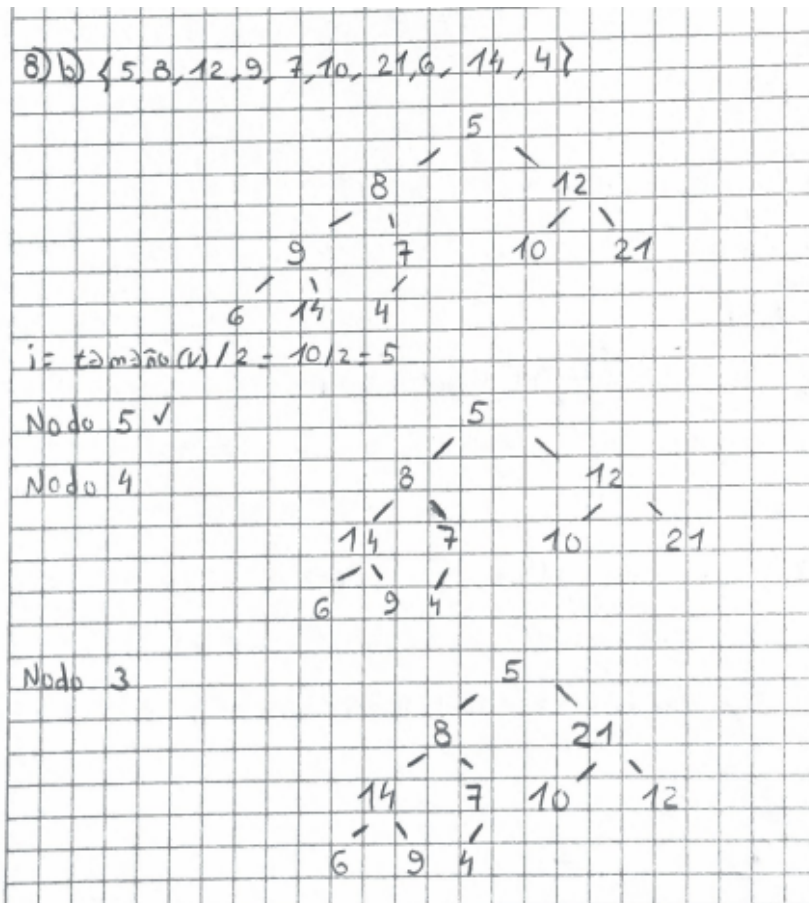


{15, 18, 33, 140, 90, 40, 500 | 13, 12, 11, 10, 7, 2, 1}



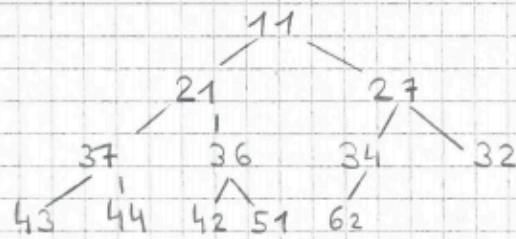
Ejercicio 8



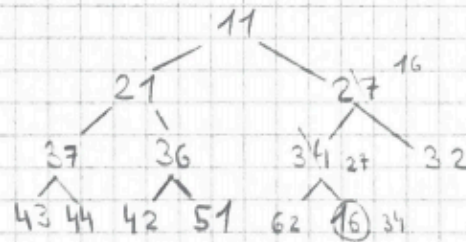


Ejercicio 9

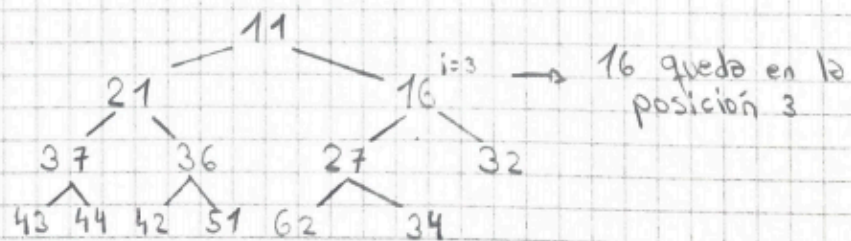
9) MinHeap



Insert 16

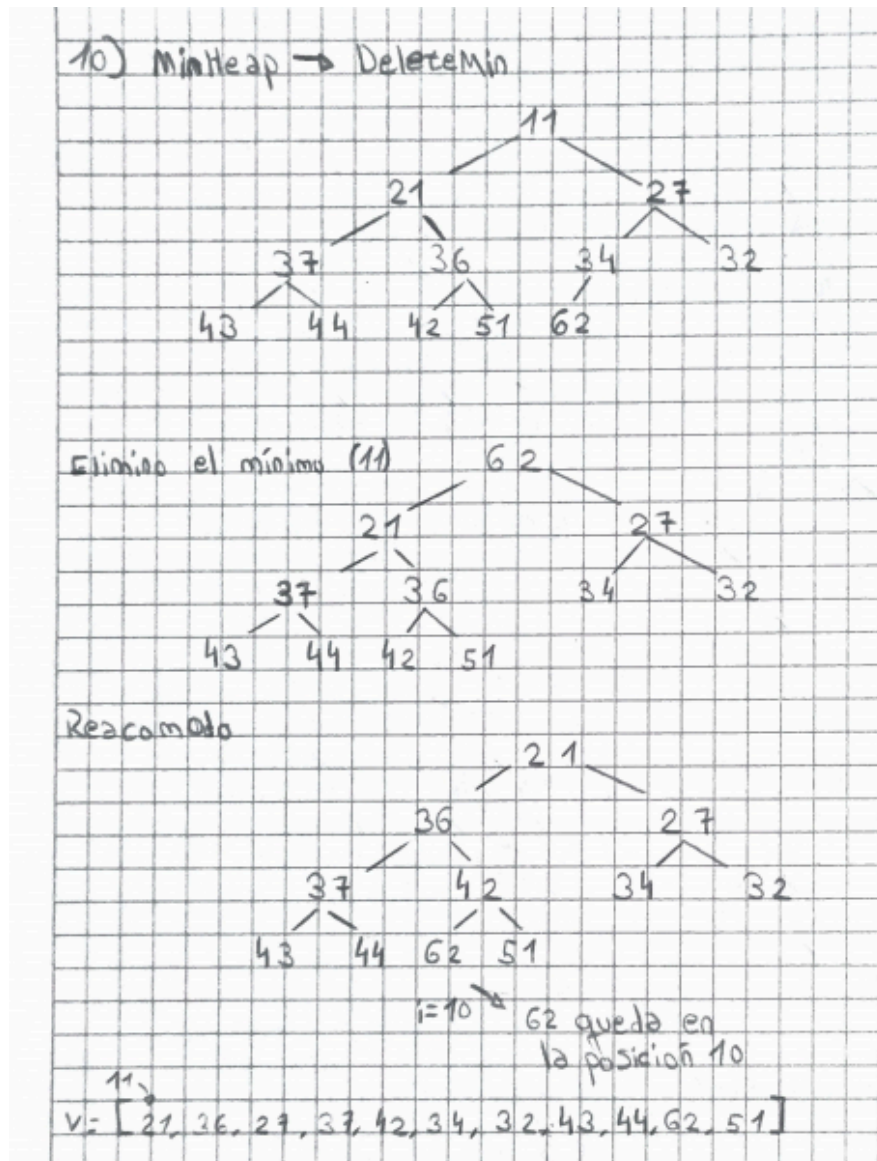


Reacomodo



El 16 queda en la posición 3.

Ejercicio 10

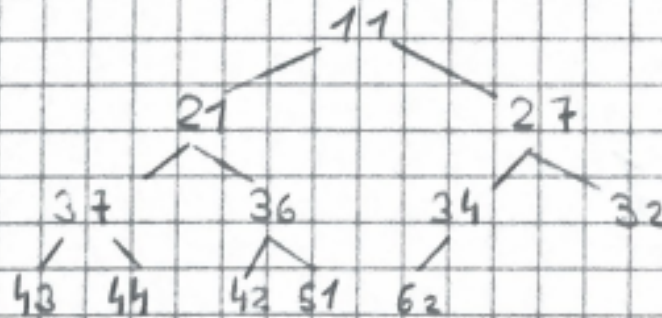


El 62 queda en la posición 10.

Ejercicio 11

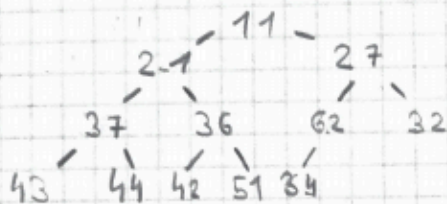
- Hacer BuildHeap para construir la MaxHeap y después aplicar la fase de ordenamiento (extraer y reordenar) para construir el arreglo de forma creciente (ascendente).
- Ya es una MinHeap por lo que no es necesario hacer BuildHeap. Se debe aplicar la fase de ordenamiento (extraer y reordenar) para construir el arreglo de forma decreciente (descendente).

11) Ordenar en forma creciente con HeapSort
 convertir a MaxHeap con BuildHeap



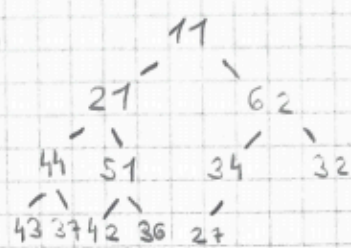
$$V[i] = i/2 = 12/2 = 6$$

1 Nudo 6 ¿34 > 62? NO

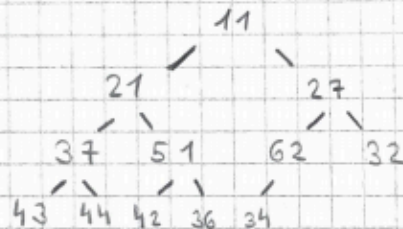


4 Nudo 3 ¿27 > 62? NO

¿27 > 34? NO

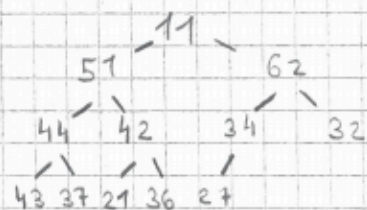


2 Nudo 5 ¿36 > 51? NO

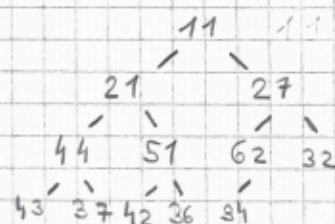


5 Nudo 2 ¿21 > 51? NO

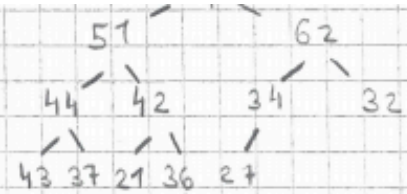
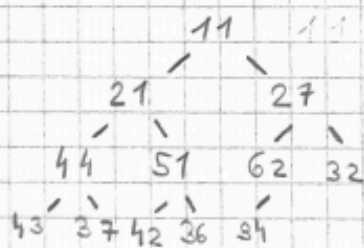
¿21 > 42? NO



3 Nudo 4 ¿37 > 44? NO



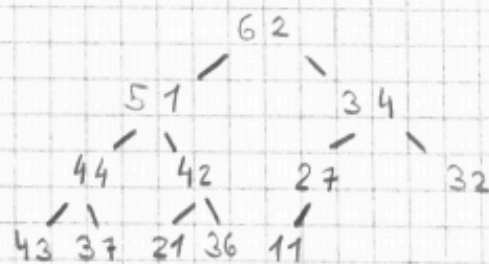
3. Nodo 4 ¿ $37 > 44$? NO



• Nodo 1 ¿ $11 > 62$? NO

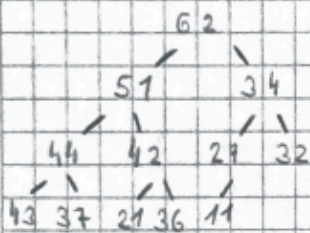
¿ $11 > 34$? NO

¿ $11 > 27$? NO



$V = [62, 51, 34, 44, 42, 27, 32, 43, 37, 21, 36, 11]$

Extraer y ordenar



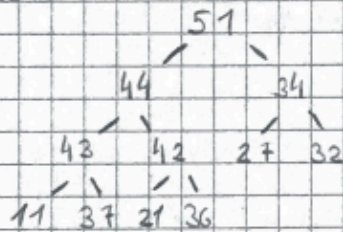
En todos los pasos voy a hacer lo mismo:

1. Intercambiar el primero con el último

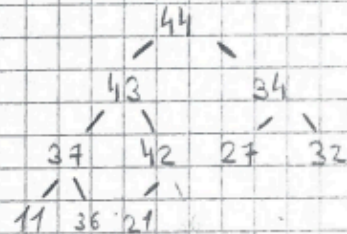
2. Decrementar el tamaño de la Heap

3. Reacomodar la nueva raíz

Paso 1

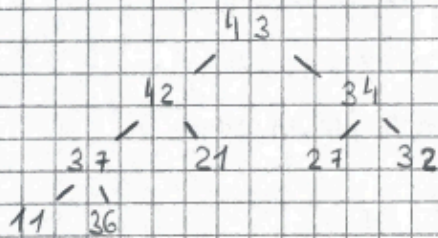


Paso 2



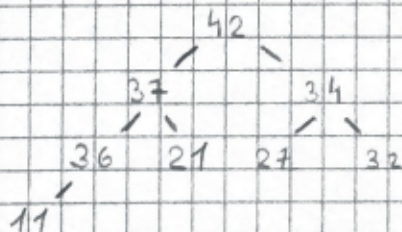
[51, 44, 34, 43, 42, 27, 32, 11, 37, 21, 36 | 62] [44, 43, 34, 37, 42, 27, 32, 11, 36, 21 | 51, 62]

Paso 3



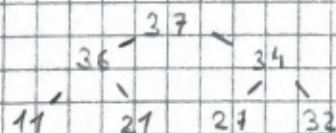
[43, 42, 34, 37, 21, 27, 32, 11, 36 | 44, 51, 62]

Paso 4

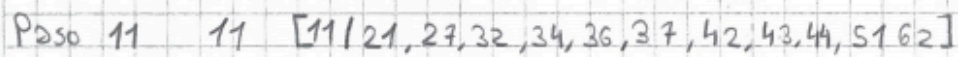
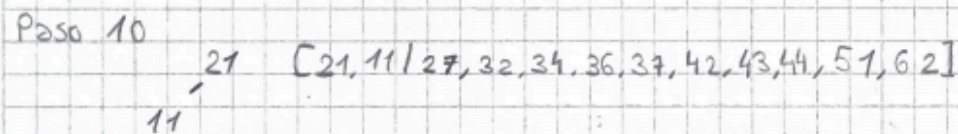
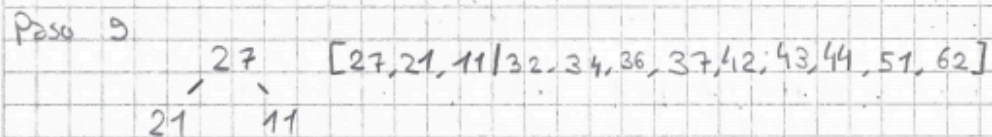
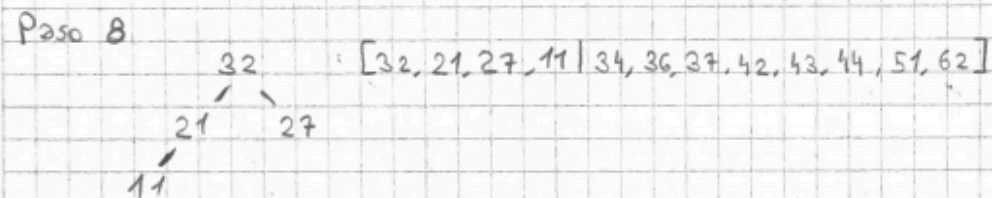
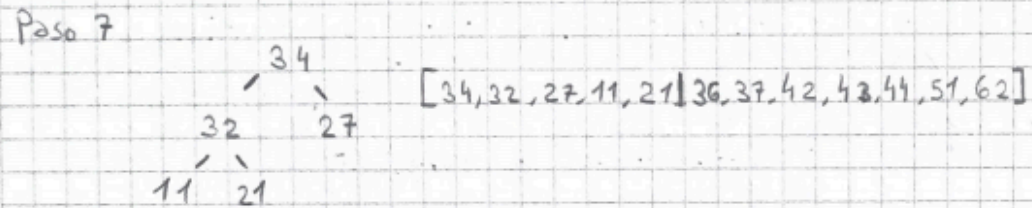
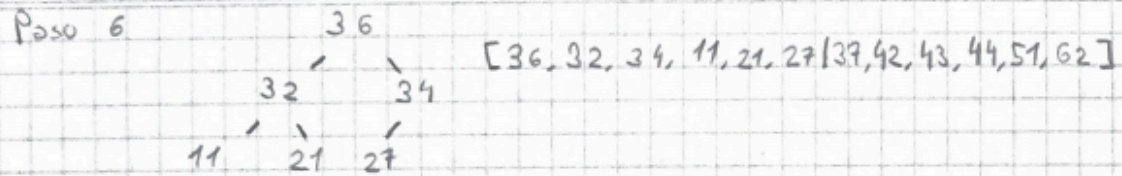


[42, 37, 34, 36, 21, 27, 32, 11 | 43, 44, 51, 62]

Paso 5



[37, 36, 34, 11, 21, 27, 32 | 42, 43, 44, 51, 62]



V = [11, 21, 27, 32, 34, 36, 37, 42, 43, 44, 51, 62]